Expert Systems With Applications xxx (2016) xxx-xxx

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com

Supergraph based periodic pattern mining in dynamic social networks

Sajal Halder [a, c], Md. Samiullah [b], Young-Koo Lee [*, c]

[a] Dept. of Computer Science and Engineering, Jagannath University, Bangladesh
[b] Dept. of Computer Science and Engineering, University of Dhaka, Bangladesh
[c] Dept. of Computer Science and Engineering, Kyung Hee University, South Korea

ARTICLE INFO

Article history:
Received 7 July 2015
Received in revised form 1 September 2016
Accepted 15 October 2016
Available online xxx

Keywords:
Periodic patterns mining
Dynamic social networks
Supergraph

ABSTRACT

In dynamic networks, periodically occurring interactions express especially significant meaning. However, these patterns also could occur infrequently, which is why it is difficult to detect while working with mass data. To identify such periodic patterns in dynamic networks, we propose single pass supergraph based periodic pattern mining *SPPMiner* technique that is polynomial unlike most graph mining problems. The proposed technique stores all entities in dynamic networks only once and calculate common sub-patterns once at each timestamps. In this way, it works faster. The performance study shows that *SPPMiner* method is time and memory efficient compared to others. In fact, the memory efficiency of our approach does not depend on dynamic network's lifetime. By studying the growth of periodic patterns in social networks, the proposed research has potential implications for behavior prediction of intellectual communities.

© 2016 Published by Elsevier Ltd.

1. Introduction

Periodic pattern mining in dynamic social network is a problem of great importance. It is relevant to many real applications such as in the area of human societies and behavior analysis (Chapanond, Krishnamoorthy, & Yener, 2005; Diesner & Carley, 2005), wild animal communities behavior matching (Fischhoff et al., 2007), social network analysis (Wasserman & Faust, 1994), and the mobile cell users behavior analysis. Monthly e-mail newsletters, yearly family reunions, monthly banking information, reports, weekly organizational meeting and birthday wishes are periodic significant patterns which are usually neglected in the study in the enormous collection of public interactions data in dynamic social networks. However, these periodic patterns are most interesting and they convey very meaningful information yet these patterns are often-infrequent. Periodic behavior represents stable interactions among animals (Fischhoff et al., 2007; Zheng & Jiang, 2012) or is used in tracking devices (Juang et al., 2002), these are qualitative interest among them. Thus, periodic patterns can predict future behavior by virtue of repeating tendencies such as ubiquitous applications (Eagle & Pentland, 2006) and sequential database applications (Yang, Hong, Chen, & Lan, 2013; Nishi, Ahmed, Samiullah, & Jeong, 2013).

Periodic pattern mining in dynamic network has been introduced by Lahiri and Berger-Wolf in Lahiri and Berger-Wolf (2008); 2010). Their proposed single pass *PSEMiner* algorithm finds periodic patterns in polynomial time. In this process, a pattern tree is created that maintains all periodic patterns. At each timestamp, the whole pattern tree is traversed and many unessential tree nodes are created that are intensely time consuming. Another algorithm *ListMiner* has been proposed by Apostolico, Barbares, and Pizzi (2011), which solves unessential tree node creation problem and speed up *PSEMiner* algorithm. In this method, less number of traversing is needed because dynamic graphs are partitioned by periodic value. At each partition, it creates a list and each list node represents a unique periodic pattern. The approach is quite faster than previous one. However, the number of partitioning list nodes has been massive and it creates nodes when any interaction is changed among the graph by time. It stores all redundantly common interactions, which are time and memory consuming. Previous two techniques *PSEMiner* and *ListMiner* store separate graphs if only one entity among a large graph is changed by time. This process stores redundant information, which requires tremendous time and memory. Hence, we need time and memory efficient periodic pattern mining technique.

Therefore, if there is any technique which updates only these entities that is changed by time not the entire graph that will be time and memory efficient. By this concept, we consider supergraph that stores all common and uncommon entities (vertices and edges) only once. We can find these entities which are identified to be updated by common entities computation that needs once common subgraph computation. This process is time efficient because previous approaches need multiple times common entities computation. In this paper, we propose supergraph based periodic patterns mining algorithm *SPPMiner* that stores all entities' periodicity individually and finds periodic entities. After finding periodic entities, we combine them and create periodic patterns. We discover interesting interactions are closely related patterns among large social graph networks.

The paper proposes some contributions. The main contributions of the paper are discussed as follows:

* Corresponding author.
Email addresses: sajal@cse.jnu.ac.bd, sajal@bsmrstu.edu.bd, sajal_csedu01@yahoo.com (S. Halder); samiullah@cse.univdhaka.edu (Md. Samiullah); yklee@khu.ac.kr (Y-K Lee)

http://dx.doi.org/10.1016/j.eswa.2016.10.033
0957-4174/© 2016 Published by Elsevier Ltd.

UNCORRECTED PROOF

- We propose single pass supergraph based periodic pattern mining *SPPMiner* technique that is polynomial unlike most graph mining problems.
- The proposed technique stores all entities of the dynamic network only once and calculates common sub-patterns only once at each timestamps which is faster than other techniques.
- We show that *SPPMiner* method is time and memory efficient compared to others. The memory efficiency of our proposed technique does not depend on the life time of the dynamic networks.
- We discover some significant group of populations in large social dynamic networks.

The remaining part of this paper is organized as follows. In Section 2, the related works are described while some preliminary concepts are discussed in Section 3. Problem definition has been discussed in Section 4. Proposed methodology has been described in Section 5. The effectiveness, efficiency and scalability of the proposed method are shown in Section 6. We have discussed the applicability of our proposed algorithm in solving problems of various real-life domains in Section 7. In Section 8, we concluded the paper with a direction of future work.

2. Related work

There have been a number of recent studies in periodic pattern mining. These periodic pattern mining researches are structured and unstructured. For example, Ozden, Ramaswamy, and Silberschatz (1998) proposed cyclic association rules, Han et al. first introduced partial periodic pattern mining algorithm in time-series databases (Han, Dong, & Yin, 1999) using partial pattern mining properties such as Apriori property (Agrawal, Imieliński, & Swami, 1993) and the max-subpattern hit set property. Tanbeer, Ahmed, Jeong, and Lee (2009) proposed discovering periodic frequent patterns in transactional databases using Ma and Hellerstein (2001) proposed a similar, Apriori based method containing two level wise algorithms in unknown period value. Yang, Wang, and Yu (2003) introduced a new asynchronous type of periodic pattern mining algorithm to find all patterns within coverage range of data sequence with maximum number of disruption allowed. Huang and Chang (2004) proposed another asynchronous method which validates segment and sequence through a minimum number of repetitions of patterns. Using probabilistic model, Yang, Wang, and Yu (2004) proposed an efficient algorithm, InfoMiner, which mines surprising patterns and associated subsequences based on the information gained. Yin, Cao, Han, Zhai, and Huang (2011) also proposed probability based latent periodic topic analysis in text databases. All of these proposed periodic pattern methods deal with unstructured data such as a sequence or multiple sequences.

Dynamic networks, which are the best example of structured data, represent a sequence of graphs over times. Graph vertices represent the populations and graph edges express the interactions among populations. In this perspective, mining periodic patterns in a dynamic network, Lahiri and Berger-Wolf (2008, 2010) proposed *PSEMiner* algorithm. This algorithm mines periodic patterns in dynamic network in polynomial time, unlike many related subgraph and itemset mining problem. This *PSEMiner* process creates a pattern tree, which maintains all periodic subgraphs seen up to timestamp t, and it tracks periodic or future periodic subgraphs. Every time the entire pattern tree is traversed and a common subgraph between current graph and tree node is created. These common subgraphs indicates a unique tree node. In this process, a number of common subgraphs is created that is useless. Projected timestamps based *ListMiner* (Apostolico, Barbares, & Pizzi, 2011) solves unessential tree node creation problem and speed up periodic pattern mining. This method maintains some list structure which indicates projection $\pi_{p,m}$ list, where p is period and $m = t \bmod p$. This approach is T times faster than the previous approach because it creates less number of lists nodes and needs less traversing.

However, the number of list nodes has been yet large and stores redundantly common subgraphs that are memory and time consuming. The number of list depends on maximum period $(P_{\max})$ because the process mines all periodic patterns up to $P_{\max}$ and the number of list nodes in the list depends on entities set which is a time consuming process.

Therefore, these facts motivated us to propose a periodic pattern mining method that overcomes the limitations of existing works. Our proposed method mines periodic pattern among dynamic network entities (vertices and edges) by calculating only once common subgraph and stores all entities only once. This process is time and memory efficient.

3. Preliminaries

Dynamic network is a representation of interactions among a set of unique populations that changes from time to time. Let, $V \in N$ represent the set of populations. Interactions among populations may be directed or undirected are supposed to have been recorded over a period of isolated time intervals. We use natural time interval quantization, such as one day or one hour which depends on the dataset. The only essential is that a time interval represents a meaningful amount of real time, where the periodicities of mined patterns will be set of chosen time interval.

Definition 1

Dynamic Network: The interactions among populations in the network are changed by time called dynamic networks. We can define a dynamic network $DN = \langle G_1, G_2, \ldots, G_T \rangle$, where $G_t = (V_t, E_t)$ is a simple interaction E_t among populations $V_t \in V$ at timestep t. Interactions and populations denoted as follows. These interactions and populations are called entities in the rest of the paper.

(i) $l(v_i)$ is the unique label of population $v_i \in V_t$.

(ii) Interaction between v_i and v_j represent as $(v_i, v_j) \in E_t$ where $l(v_i) < l(v_j)$.

Fig. 1 shows an example of a dynamic network with five timestamps. Definition 1 reduces the high computational complexity of many algorithmic tasks on graph databases.

Definition 2

Supergraph (SG): Supergraph is a graph database that compact several graphs into one graph with the common entities of the graphs being stored only once.

Suppose a graph has 4 population (A, B, C, D) in the Fig. 2. At first timestamps population interactions are (A-B), (B-C) and (C-D), and second timestamps population interactions are (A-B), (A-D), (B-D) and (C-D). According to the Definition 2 the supergraph compacts two graphs (G_1 and G_2) interactions (A-B), (A-D), (B-C), (B-D) and (C-D), where common interactions are stored only once.

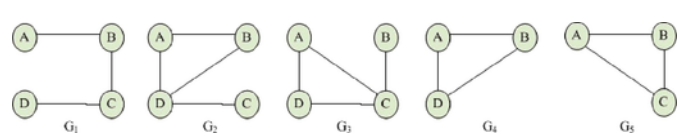


Fig. 1. Dynamic graph structure.

UNCORRECTED PROOF

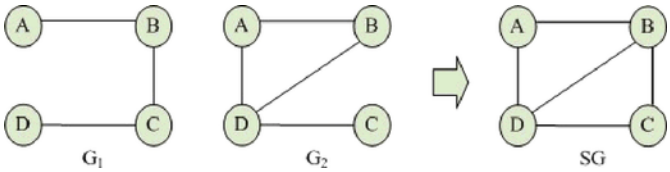


Fig. 2. Supergraph.

Property 1

Graph Representation: A graph $G = (V, E)$ with unique vertex labels is represented by two unique vertex labels in N where N is a natural number. Two vertex labels indicate one edge between them.

Since each vertex is uniquely expressed by its label, each edge is also expressed by the interaction between two unique vertices. This allows each vertex to be labeled as a unique integer, even across different graphs over the same vertex set. Two graphs will be same if their vertex label sets are same and its corresponding edge means connected vertices label sets are same. Fig. 3(a) shows two graphs G_1 and G_2 where vertices sets are same, but edges sets are different thats why they are not same. Connectivity information remains unchanged in this representation. Each vertex is connected with other vertexes as connected edges.

Property 2

Pattern Testing: The measurement of pattern testing whether G_1 is a pattern of G_2 or vice versa can be done by checking unique vertex label representation sets and their corresponding connected edge label of G_1 is subset of G_2 or vice versa.

Property 3

Maximum Common Pattern (MCP): The MCP between two graphs is defined by the common vertices label and their corresponding common connected vertices label. It may be connected or disconnected patterns.

From Fig. 3(a), we find the maximum common pattern between G_1 and G_2 containing all unique vertex labels < *A, B, C, D* > and two common connected edge (*A, B*) and (*C, D*). Following Fig. 3(b) shows the MCP representation with unique vertex label and Fig. 3(c) depicts MCP structure.

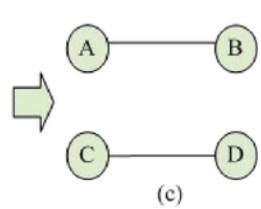
Property 4

G_1		G_2	
Vertices	Edges (Vertex Label)	Vertices	Edges (Vertex Label)
A		A	
B	A	B	A
C	B	C	
D	C	D	A, B, C

(a)

Vertex Label	Edge (Vertex Label)
A	
B	A
C	
D	C

(b)



(c)

Fig. 3. (a) Graph representation with unique vertex label, (b) MCP representation with unique vertex label and (c) MCP pattern.

Hashing: Since the unique vertex labels set is represented by N has a global ordering. Edges are mentioned by two unique vertex labels that why have to use two hashing for indicating vertices and edge sets.

For mining periodic patterns, time indication is particularly important, because periodic patterns depend on period and starting time. A pattern occurring number is also essential for counting support.

Definition 3

Periodic Support Set: Given a dynamic network *DN* of *T* time interval and any pattern F = (V, E). The periodic support set $S_p(F)$ of pattern $F \in DN$ is the set of all time interval *t*, which start on t_i time and repeating every p time interval where F is a pattern of DN, which denote $F \subseteq G_{t_i}$. The representation of support set $S_P(F) = (t_i, p, s) = \{t_i, t_i + p, \ldots, t_i + p(s-1)\}$, such that $\forall t_i \in S_p(F) \leftrightarrow F \subseteq G_{t_i}$ and neither $G_{t_i - p}$ nor $G_{t_i + ps}$ contains *F* as a pattern.

From the Fig. 1 we observed graph interaction (A-D) subsumes in graph G_2, G_3 and G_4. According to the Definition 3 its starting time is 2 and repeats 1 time interval and its support is 3 that means $S_p(A - B)$ = (2,1,3) which occurs {2, 3, 4} timestamps. We also show that interaction is neither $(A - D) \subseteq G_1$ nor $(A - D) \subseteq G_5$.

Definition 4

Frequent Pattern: A periodic pattern *F* is a frequent pattern if its support exceeds a user defined minimum support threshold value σ.

Definition 4 is the formulation that is derived from the well known frequent pattern mining problem satisfying the downward closure property.

Definition 5

Closed Pattern: For any pattern F = (V, E) in a dynamic network DN of T time interval is closed pattern if it is maximal for its support set. In other words, while the support of F is maintained, no vertex or edge can be added.

There is a difference between frequent closed subgraph support set and closed subgraph support set. A single subgraph F can have multiple periodic pattern support set to allow disjoint and overlapping periodic behavior. Thus, we require extraction of all periodic subgraph embedding, rather than just the periodic subgraphs. According to Definition 6, pattern carries multiple types of periodic information. Suppose a periodic pattern with period p, it is also periodic at every multiple of p, which depends on only the threshold value. If they are frequent, they will be the output as periodic patterns and redundancy problem will be occurred.

Definition 6

Periodic Pattern (PP): Given a dynamic network *DN* and an arbitrary pattern *F*. The periodic pattern (PP) is a pair of < *F*, $S_p(F)$ >, where F is closed pattern over a periodic support set $S_p(F)$ with $|S_p(F)| > \sigma$ and $S_p(F)$ is maximum for F.

Mining parsimonious periodic patterns (PPP) is a well-designed solution to the redundancy of general periodic patterns. Since it captures all the information and produces a small number of output results that is defined in the Definition 7. It maintains subsumption property in Property 5.

Property 5

UNCORRECTED PROOF

Subsumption: Two periodic patterns F_1 and F_2, their support set $S_p(F_1) = (t_1, p_1, s_1)$ and $S_p(F_2) = (t_2, p_2.s_2)$. $S_p(F_1)$ contains or subsumed $S_p(F_2)$ if and only if the following conditions hold.

i. $F_2 \subseteq F_1$

ii. $t_2 \geq t_1$

iii. $p_2 \bmod p_1 = 0$ *and* $p_1 < p_2$

iv. $t_2 + p_2(s_2 - 1) \leq t_1 + p_1(s_1 - 1)$

v. $(t_2 - t_1) = p.k$ *for some integer* $k > 0$.

Definition 7

Parsimonious Periodic Pattern (PPP): A periodic pattern (PP) that is not subsumed by the others PP is a parsimonious periodic pattern.

Suppose consider a DN in which interaction $A - B$ occurs at timestamps ⟨2, 4, 6, 8, 10⟩. If minimum support is 3 and we consider period is 2 then we get periodicity $P_1 = \langle 2,4,6 \rangle$, $P_2 = \langle 4,6,8 \rangle$, $P_3 = \langle 6,8,10 \rangle$, $P_4 = \langle 2,4,6 \rangle$, $P_5 = \langle 2,4,6,8 \rangle$ and $P_6 = \langle 2,4,6,8,10 \rangle$. We observe that periodicity P_6 subsumed periodicity P_1, P_2, P_3, P_4 and P_5. On the other hand, if period is 4 we get periodicity $P_7 = \langle 2,6,10 \rangle$ which also subsumed by P_6. In the above discussion, we get the interaction $A - B$ is a parsimonious pattern at periodicity P_6. Others periodicities are non-parsimonious even though they are periodic. With non-parsimonious patterns duplicates of each true periodic pattern would be reported for a fixed number of multiples of either 2 or 4. To reduce the duplicate's periodicity of a pattern we mine parsimonious periodic patterns.

Definition 8

Periodic Pattern Mining Problem: Given a dynamic network DN and a minimum support threshold σ, the Periodic Pattern Mining Problem is to mine all parsimonious periodic patterns in *DN* those satisfy the minimum support threshold.

4. Problem statement

Suppose the support of periodic pattern F is $S(t, p, s)$ that means pattern F occurs at first t timestamp and continues p period interval upto s times. We define $S(t, p, s) = \{t, t+p, \ldots, t+p(s-1)\}$, where $t \geq 0$ and $p, s \geq 1$. A pattern F may have several periodic supports sets. Not all of those periodic supports are parsimonious. Let pattern F occurred in two periodic set $P_1 = S(t_1, p_1, s_1)$ and $P_2 = S(t_2, p_2, s_2)$. We say that P_1 subsumes P_2 if and only if $S(t_2, p_2, s_2) \neg \subseteq S(t_1, p_1, s_1)$. That implies P_1 is a parsimonious periodic pattern for pattern F but P_2 is not parsimonious.

For example, in Fig. 1 interaction between A and B occurs < 1, 2, 4, 5 > time interval graph. Suppose minimum $\sigma \geq 2$ then we find periodic patterns support $P_1 = (1,1,2)$, $P_2 = (1,3,2)$, $P_3 = (1,4,2)$, $P_4 = (2,2,2)$, $P_5 = (2,3,4)$ and $P_6 = (4,1,2)$. On the other hand, interaction between C and D occur < 1, 2, 3 > time interval. We find periodic patterns $P_7 = (1,1,2)$, $P_8 = (1,2,2)$, $P_9 = (2,1,2)$ and $P_{10} = (1,1,3)$. Periodic patterns P_7, P_8 and P_9 are subsumed by P_{10} that called parsimonious periodic pattern. Our goal is finding all parsimonious periodic patterns in dynamic networks.

5. Supergraph based periodic patterns mining

This section presents the main contribution of this paper; the design and development of *SPPMiner*, a supergraph based periodic pattern mining technique that improves the worst-case time and space complexity of *PSEMiner* (Lahiri & Berger-Wolf, 2010, 2008) and *ListMiner* (Apostolico, Barbares, & Pizzi, 2011) algorithms. The key idea of the proposed method is to reduce the number of common pattern computation. It needs only one maximal common pattern (MCP) calculation at each time interval graph G_t. On the other hand, all common and uncommon patterns entities (vertices and edges) among dynamic social networks are stored only once, which requires less memory and capable of avoiding redundant information storage. The following sub-sections formalize the concepts and present detailed descriptions of the proposed algorithm.

5.1. SPPMiner

Now we propose our algorithm *SPPMiner* for mining all periodic pattern in dynamic networks. At first, we start by describing the most basic form of the algorithm, which explore periodic patterns. Then two pruning methods have been applied that discards non-closed and non-parsimonious periodic patterns. After pruning steps we will find parsimonious periodic patterns. The architecture of *SPPMiner* has been shown in Fig. 4.

SPPMiner works on following steps: at each time interval dynamic graph network is read, then a supergraph embedding of all network entities seen up to time interval t. This supergraph maintained two kinds of data structure for each entity. One is timeset, which stores the active time of an entity. Another one is descriptor list, representing entity periodicity information. Once entities cease to be periodic, they are flashed from the supergraph and inserted into a periodic hash table as a periodic entity. Each entity stores only three types of information, not all time interval information that is memory and time efficient. Periodic hash table is one kind of data structure that stores entities based on period and starting time. If entities period and support are same then we combined together to build periodic patterns. Then we use two kinds of pruning properties: non closed and non parsimonious patterns pruning and find parsimonious periodic patterns. If its super pattern with the same descriptors does not exist, it is closed. If its support does not subsume by other descriptors, it is considered as parsimonious descriptor and pattern is the parsimonious pattern for the descriptor. The algorithm parameters, data structures and the proof of correctness describe one by one.

5.1.1. Parameters

Our algorithm is online periodic pattern mining single pass and efficient algorithm that mines parsimonious periodic patterns in a dynamic network. It needs two parameters as follows.i. Minimum support threshold σ (3 default)ii. Maximum period $P_{\max}$ (40 default)

The online algorithm bound the maximum period of supergraph entities. When entities create new descriptor, it generates maximum $P_{\max}$ number of descriptors and the minimum support threshold is used to indicate whether flushed entity is periodic or not?

5.1.2. Data structure

As the algorithm looks into the stream graphs, it maintains three kinds of data structures: timeset, descriptor set and periodic pattern hash table.

Descriptor

A descriptor D is one kind of data structure that represents periodic support set (t_i, p, s) for any entity, where starting time t_i, period p and supports. Descriptor D for an entity E is live if its expected time $(t_e = t_i + ps) \geq$ current time t_c and entity E is present at G_t. A descriptor that is not live considered as a periodic entity if its support satisfies the minimum threshold value.

In supergraph, each entity contains a set of descriptors that is periodic or to be periodic in the future time interval. Generating future

UNCORRECTED PROOF

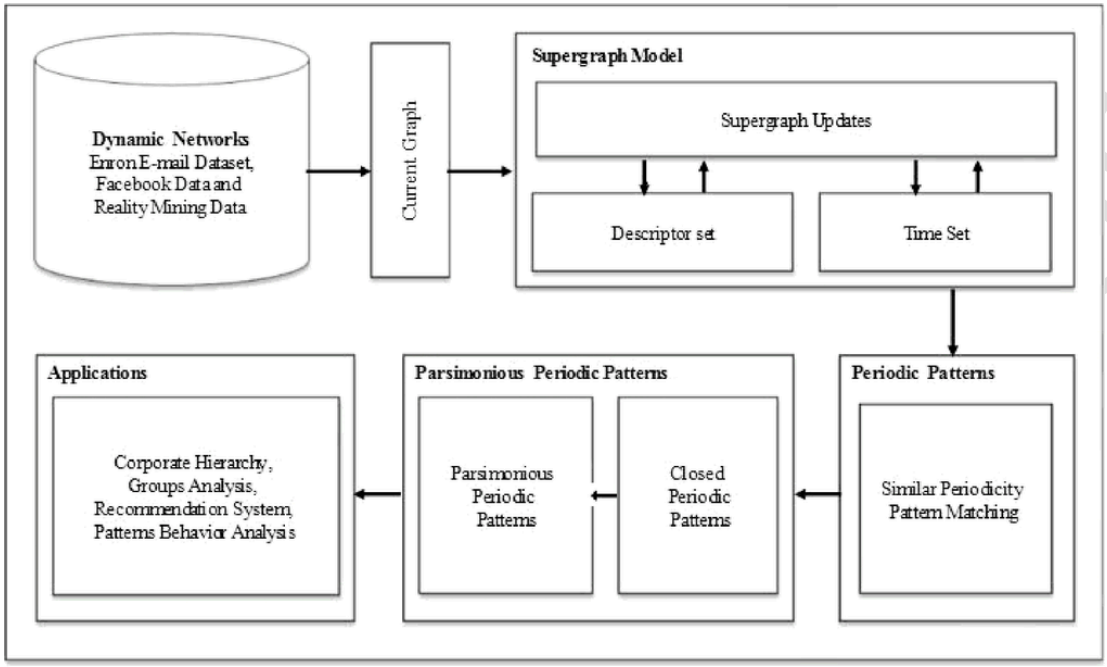


Fig. 4. Supergraph based periodic patterns mining architecture.

periodic descriptors, each entity needs to store occurrence time in the previous time interval.

Time Set (TS)

Time set is one kind of data structure that stores active time of the entity.

Entity time set length depends on maximum periodic value. Here we consider $P_{\max}$ as maximum period. Following lemma can be described regarding the size of TS.

Lemma 1

The maximum size of TS for any entity is $P_{\max}$.

Proof

Periodic entity repeats every p period time interval. Descriptor indicates entity periodicity information and support. If one entity appears in the current graph, it updates descriptors of common supergraph entity. If descriptors expected time is equal to the current time, then update these descriptor's information. Otherwise, the current entity generates a set of descriptors that would be periodic next time and stores the current time into timeset (TS). For new descriptor, maximum period is $P_{\max}$ because if it appeared in the previous and live then we can say it already exists in updated descriptors. Thus, entity time set size is maximum $P_{\max}$ and it stores all periodic time exclusive of missing any essential information.

Periodic Pattern Hash Table

Supergraph flushed out periodic support set for each entity. Our main goal is finding periodic patterns. In support of that principle, we need to add flushed entity descriptor to generate patterns. Hash table is especially efficient for this variety of structures. We have used starting position, period and support combination as a key value of a hash table and have stored corresponding entities into patterns.

5.2. Description of algorithms

The supergraph information update process is the core part of our periodic patterns mining technique will be describe here. Initially the supergraph (SG) is empty. At timestamp t, graph G_t is read. The common entities between SG and G_t are updated into SG. Entity updates timeset and descriptor set including addition, deletion and modification. Descriptors are flushed at deletion step. If descriptors support is greater than the minimum threshold value, the entity is periodic. The process of time t is completed, ensuring all uncommon entities in current graph G_t includes in supergraph. The entire process is partitioned following three parts. We describe each part in the following subsections in details.

5.2.1. Update common entities in supergraph

The supergraph entities timeset and descriptor sets are updated when entities are active. An entity appears in graph G_t is active at timestamp t otherwise it is inactive. Each timestamps graph G_t, the supergraph should be updated with common and uncommon entities. Let $F = SG \bigcap G_t$ be the maximal common subgraph of supergraph SG and G_t.

Timeset Update

Entity timeset presents active timestamps of the entity. Each timestamp t graph represented by graph G_t. However, SG compacts a set of time series graphs that is why active entity timestamps should be stored. These timestamps are needed to generate entity periodicities, which are not periodic at this moment, but would be periodic in the future. *SPPMiner* stores only last $P_{\max}$ number of periodic information instead of all periodic time information because entities information appears reiteration at some periodic time interval. If timeset size is maximum, which is defined by Lemma 1, then delete timestamps using first in first out (FIFO) order.

UNCORRECTED PROOF

Descriptors Creation

When entity $E \in$ pattern F, is active at current time t, it can create some new periodic descriptors which are not currently periodic. Using entity timeset, it creates future periodic descriptors. Descriptor period indicates the difference between previously storing timeset time and the current time.

Lemma 2

Each entity E creates maximum $P_{\max}$ descriptors at timestamps t.

Proof

New descriptor indicates entity periodicity that would be periodic at some point in the future. In our method, we define maximum period $P_{\max}$. Thus, entity can generate descriptor that period is 1 upto $P_{\max}$. So, the maximum number of descriptor at any time step is maximum $P_{\max}$.

Suppose an entity timeset $TS = \{1, 2, 3, 4]\}$ and current time is 5. It can create four descriptors. These descriptors periodic support sets are like (1,4,2), (2,3,2), (2,2,2) and (4,1,2), where first element is starting position, second one is period and last one is support value. *TS* can store maximum $P_{\max}$ time set that result each entity creates maximum $P_{\max}$ descriptors at a single timestamp.

Descriptors Update

If entity $E \in F$, i.e. entity E is active at timestamp t, Let D is a descriptor of $E \in SG$ and $te = t_i + ps$ be the next expected time for D. If $te = t$, then D has appeared where it was expected and created a new descriptor $D' = D$. Time t is added to D' support to ensure temporal maximal. If the support of D is greater than or equal to σ then remove from supergraph entity E and stores in the periodic hash table. The following property is maintained at update state.

Lemma 3

Entity E updates maximum $min\left(t, P_{\max}\right)$ descriptors at timestamps t.

Proof

Descriptor D updates when expected time is equivalent to current time. For each period p, there are exactly p number of descriptors, these should be updated at time t. These descriptors are defined by a period and phase such as p mod m where $0 \le m < p$. If period $p = 4$ then descriptors period and phase pairs (1,0), (2,0), (3,1) and (4,0) should be updated. The maximum value of period p is $P_{\max}$. Therefore, the number of descriptors at any time t will be $min\left(t, P_{\max}\right)$.

Descriptors Deletion

Suppose entity E has descriptor D that has expected time $te < t$, then D has not appeared when expected and is no longer live. If it's support $\ge \sigma$, it will be stored in a periodic hash table as a periodic entity and will remove D from $E \in SG$.

Lemma 4

At timestamps t, average $2 * P_{\max}$ number of descriptors have been deleted from entity E.

Proof

Lemmas 2 and 3 show that in time t, maximum $P_{\max}$ number of descriptors have been created and updated. These means maximum $2 * P_{\max}$ number of descriptors are added at each timestamp. The number of deleted descriptors will be same as added descriptors in entire timestamps. However, the numbers of deleted descriptors are not fixed and there are no upper binds because at the last step, numerous descriptors should remain alive. In this case, we can estimate the average number of descriptors deletion at each timestamps and it will be same as creating and updating number that is $2 * P_{\max}$.

Each timestamps t, entity E creates new descriptors, updates descriptors and finally deletes descriptors that may be periodic or not. Therefore, the number of descriptors at entity E maintain the following property defined in Lemma 5.

Lemma 5

The maximum number of descriptor of any entity at time t is $P_{\max}^2$.

Proof

The maximum number of descriptors of any entity depends on size of entity time set. Lemma 2 declared that every timestamps entity creates $P_{\max}$ future behavior descriptors, including period 1 up to $P_{\max}$. If same periodic descriptors exist, then update these descriptors by creating new descriptors and delete old descriptors. Each periodic descriptor can be started at any position of phase m, where $0 \le m < P_{\max}$. Thus, the maximum number of descriptors of entity is $P_{\max}^2$.

Entities Deletion:

An entity appears in G_t is active otherwise inactive. Storing and processing long time inactive entity in supergraph is inefficient because it needs memory and more *MCP* computation time for mining common patterns between supergraph and current graph G_t.

Lemma 6

Inactive entity survives maximum $P_{\max}$ time in a supergraph.

Proof

For each entity, maximum $P_{\max}$ periodic descriptors are contained. When one periodic expected time is passed, those periodic descriptors are removed. After $P_{\max}$ time there are no descriptors in entity if the entity does not become active within this period. When an entity descriptor set is empty up to $P_{\max}$ time, we said this entity is dead. Then we can delete these entities from supergraph because there is no chance to be periodic within a bounded period if it appears in future it will create new periodic descriptors.

Suppose entity E, appears in 1 and 6 timestamps and $P_{\max} = 3$ then we can delete entity at time step 4 because it already reached dead entity. Next time if it appears, it will be considered as a new entity and stores newly.

5.3. SPPMiner algorithm

Algorithm 1 shows the proposed *SPPMiner* algorithm. Initially supergraph is empty. At each time interval t, supergraph is updated based on current graph entities that is mentioned at line 5. However, these entities do not appear in the current graph update at line 7. If any entity remains inactive in consecutive $P_{\max}$ times, then remove it from the supergraph at line 9. There are some entities in current graph which do not exist in supergraph. These entities are merged with supergraph at line 14. Finally, we mine periodic entity at line 17 those are living at the last time interval.

UNCORRECTED PROOF

```
Data: G_1, G_2, ..., G_T: Dynamic pattern graphs at timestep 1, 2, ..., T; σ : min_sup;
Result: Periodic Pattern Sets
1  SG ← ϕ                          /* Behavior supergraph is empty                    */;
2  for t = 1 to T do
3      for ∀ E ∈ SG do
4          if E ∈ G_t then
5              Update_Entity(SG_E, t, σ, com);
               /* Update common entity                                              */;
6          else
7              Update_Entity(SG_E, t, σ, uncom);
               /* Update uncommon entity                                            */;
8              if t − E^{|ST|−1} > P_max then
9                  Remove(SG, E);                  /* Remove dead entity            */;
10             end
11         end
12     end
13     for ∀ E ∉ SG and E ∈ G_t do
14         Add_Entity(SG, E); /* Add uncommon entity                               */;
15     end
16 end
17 Mine Last Timestep_Patterns(SG, σ);
   /* Mining periodic behaviors at last timesteps                                   */;
```

Algorithm 1. SPPMiner ($\{G_1, G_2, ...G_T\}$, σ).

5.3.1. Entity update algorithm

The core part of the *SPPMiner* algorithm is Update_Entity() procedure. This procedure tracks entities periodicity and support information those are significantly important for mining periodic behaviors in dynamic social networks. Algorithm 2 shows the entity updates algorithm. Entity E is updated, including descriptor set and timeset update, deletion and creation. At first we consider all descriptors of the entity E in the supergraph SG_E^D at line number 1. Each descriptor D has three values: descriptor support (*D.sup*), descriptor period (*D.period*) and descriptor update expected time (*D.te*). If descriptor expected time is equal to current time t, we create a duplicate descriptor D' at line number 2–3. Then we update D' descriptor support and expected update time in line number 4–5. At line number 5, we update supergraph entity based on update descriptor D'. Then flashed out old descriptors with the corresponding entity at line 8. These flashed entity would be periodic entity if it supports is greater than or equal to σ (minimum support) which is shown in Algorithm 3.

```
Data: SG_E: Supergraph entity E; t: current timestamp; σ : min_sup;
Result: Update supergraph entity SG_E
1  for ∀ D ∈ SG_E^D do
2      if D.te == t then
3          D' = D;
4          D'.sup = D.sup + 1;
5          D'.te = (D'.sup + 1) × D'.period ;
6          Insert_Descriptor(SG_E^D, D');
           /* Insert Update Descriptor                                              */;
7      end
8      Flashed(D, E, σ);                  /*          Delete Descriptor            */;
9  end
10 for ∀ t' ∈ SG_E^TS do
11                                  /*          Create new descriptors              */;
12     if t − t' > P_max then
13         Remove(SG_E^TS, t');                /*          Delete time t'           */;
14     else
15         D = new Descriptor(t', t − t', 2);
16         if D ∉ SG_E^D then
17             Add_Descriptor(SG_E^D, D);      /*          Add new descriptor       */;
18         end
19     end
20 end
21 Add_Time(SG_E^TS, t);               /*          Add time t to Time Set           */;
22 return SG_E                         /*          Return update supergraph entity  */;
```

Algorithm 2. Update_Entity(SG_E, t, σ).

The other data structure of an entity is timeset (*TS*) that stores last $P_{\max}$ times when the entity was active. At timestamps t, entity may be periodic or will be periodic at some point in the future. If current time (t) and *TS* storing time (t') difference is greater than $P_{\max}$ then *TS* storing time (t') is deleted at line 13. Generating future periodic descriptor set, the algorithm creates new descriptors at line 15. If descriptors do not exist in entity, then add descriptor at line 16–17 and finally add current time t in timeset *TS* at line 21. This algorithm returns update entity to *SPPMiner* algorithm and flushed out periodic entity.

Algorithm 3 shows flashed procedure for descriptor D with entity E. If the descriptor support *D.sup* is greater than minimum support σ then it is considered as a periodic entity. All those entities are combined based on its start timestamps, period and support value. For combining entities, we create a periodic hash table key based on starting position, period and support in line 1. Sometimes descriptor creates new patterns or updates. If a hash key exists in the periodic table then find out hash value and add current entity and store again based on same hash key mentioned by line 4–5 otherwise creates a new periodic pattern and store in hash table at line 6.

5.3.2. Mining periodic behaviors

Periodic hash table value indicate a periodic pattern. However, these kinds of patterns are neither closed nor parsimonious. When mining closed and parsimonious periodic behaviors, we have to maintain two basic lemmas those are defined by Lemmas 7 and 8.

Lemma 7

Let periodic pattern F support set $S(F) = (t_i, p, s)$ *then F is closed pattern if there are no* $S(F') = (t_j, p, s')$ *where* $s' > s$, $F \subseteq F$ *and* t_i *mod p is equal to* t_j *mod p.*

Proof

According to Definition 5, closed periodic patterns are those patterns which have no super pattern with same support or same pattern with larger support. Then straightforward we can find out closed periodic patterns from the periodic hash table.

In Table 1, pattern $F = (A, B), (C, D)$, $S(F) = (1, 1, 3)$ is closed because position (1, 1, 6) pattern $F' \neg \supseteq F$. On the other hand $F = (A, B), (B, C)$, $S(F) = (2, 2, 2)$ is not closed because position (2, 2, 3) pattern $F' \subseteq F$.

Using this lemma, we can prune non-closed periodic patterns thus reduce redundant information. Another kind of redundant periodic patterns exists in periodic hash table subsumed by others is defined in Property 5. Mining parsimonious periodic patterns by the following lemma puts strongly effective influence.

Lemma 8

```
Data: D: Descriptor; E: graph entity; σ:min_sup
Result: Insert into Periodic Hash
1 hkey ← HashKey(D.period, D.phase, D.sup);
  /*                                Create hashkey                                  */;
2 if D.sup ≥ σ then
3     if hkey ∈ Periodic Table then
4         PS ← findValue(PPatterns, hkey) ⋃ E;          /*          Find subgraph     */;
5         setValue(PPatterns, hkey, PS, D.start);
          /*                    Set subgraph based on support                       */;
6     else
7         setValue(PPatterns, hkey, E, D.start);
          /*                    Set Entity based on support                         */;
8     end
9 end
```

Algorithm 3. Flashed(D, E, σ).

CORRECTED PROOF

Table 1
Closed and parsimonious periodic subgraphs characteristics.

Hash Key	Pattern	Closed	Parsim
1_1_6	(A,B),(C,D)	Yes	Yes
1_1_3	(A,B),(B,C)	Yes	Yes
2_2_2	(A,B),(C,D)	No	No
2_2_3	(A,B),(C,D)	Yes	No

Let periodic pattern F support set $S(F) = (t_i, p, s)$ *then F is subsumed by* $S(F') = (\beta * \lfloor t_i/p \rfloor, \beta, s')$ *if* $F \subseteq F'$, *p mod* $\beta == 0$ *and* $s' \geq p*s$.

Proof

We can limit ourselves to the discovery of all periodic patterns of period 1. If pattern F is periodic at $S(F) = (t_i, p, s)$. It also periodic at ($\lfloor t_i/p \rfloor$, 1, s') of projection of pattern, for all $1 < p \leq P_{\max}$ and $0 \leq t_i \% p < p$. Then it also is periodic at divisor of p. If β mod $p == 0$ then β is the divisor and ($\beta * \lfloor t_i/p \rfloor$, β, s') is periodic support with pattern F'. If $F' > F$ and $s' \geq p*s$ then F is subsumed by F'.

Suppose pattern $F = (A, B), (C, D)$, $S(F) = (2, 2, 2)$ that means it occurs 2,4,6 timesteps. It may be subsumed by (1, 1, s') where $s' \geq 2*2$. That why $F' = (A, B), (C, D) = F$ $S(F') = (1, 1, 6)$ subsumed F which shows in Table 1. F also subsumed subgraph at position (2,2,3). So F is subsumed by F' and F is not parsimonious.

Algorithm 4 shows the parsimonious periodic patterns (PPPatterns) mining procedure. Each hash key mentions the periodicity of pattern F that contains periodic descriptor as a hash value. The pattern F checks if a pattern F is associated with large support and contains periodicity information with F periodicity. It checks closed and parsimonious patterns. Finally, from the algorithm we can mine parsimonious periodic patterns.

5.4. Time and space complexity

Supergraph maintaining is the vital part of our proposed algorithm. Suppose V is the vertex number of our supergraph. Then total entities are $V * (V-1)/2$ if supergraph is strongly connected. However, the worst case super graph entity is $O(V^2)$. Lemma 5 proved that every entity has maximum $(P_{\max})^2$ descriptors. Updating these descriptors, need $(P_{\max})^2$ time and updating timeset (TS) needs $P_{\max}$ time. So each timestamp requires $O(V^2)\left((P_{\max})^2 + P_{\max}\right) = O\left(V^2 (P_{\max})^2\right)$ times. This yields the total time complexity is $O\left((V^2)\, T (P_{\max})^2\right)$ when $P_{\max}$ is specified.

For every timestamp t, supergraph has maximum V^2 entity and each entity contains *TS* and descriptor sets. According to Lemmas 1 and 5, the size of time set and descriptor set is $P_{\max}$ and $(P_{\max})^2$ respectively. Therefore, the total space complexity is $O\left((V^2)\,(P_{\max})^2\right)$ that is total time independent.

```
Data: PPatterns: periodic patterns
Result: Parsimonious periodic patterns
Iterator I1, I2;
for I1 ← PPatterns.begin to PPatterns.end do
    Hash key ← I1.Key;
    Descriptors D ← I1.Value;
    parsim ← true;
    for I2 ← I1 + 1 to PPatterns.end do
        Hash key2 ← I2.Key;
        Descriptors D2 ← I2.Value;
        if key1 ⊆ key2 and D1 ⊆ D2 then
            parsim ← false;
    end
end
if parsim = true then
    Print (I1);
end
Find(phase,period,support) ← D;
Hash nKey = new Key(phase, period, sup'); where(sup' ≥ support)
if PPatterns(nKey) & (F' = PPatterns(nkey).value) ⊇ F then
    Delete.PPatterns(nKey);
end
Hash nKey = new Key(β * ⌊m/p⌋,β, sup');
                /*          where (sup' ≥ support)          */;
if PPatterns(nKey)&(F'' = PPatterns(nkey).value) ⊇ F then
    Delete.PPatterns(nKey);
end
end
```

Algorithm 4. PPPatterns(*PPatterns*).

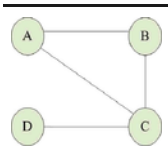
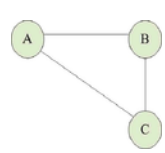
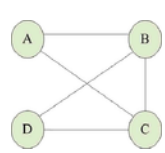
5.5. Example

Suppose the dynamic network in Fig. 1 is the input. We explain our *SPPMiner* algorithm in details systematically. It maintains supergraph update including descriptor operations and TS operations at each timestamps. Every unique time interval new periodic patterns are founded and those patters are inserted into hash table then mine parsimonious periodic patterns. In this example, we consider $\sigma = 2$.

Therefore, every timestamps supergraph is updated and periodic patterns are stored in hash table using its descriptors information, which indicates periodic essential information and satisfy minimum support. In this process, dynamic graph G_1, G_2, G_3, G_4 and G_5 updates supergraph and flashed out periodic patterns. Then finally, we find all periodic patterns those shown in Table 2.

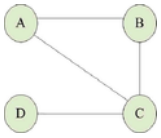
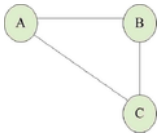
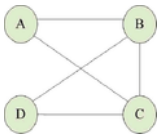
All periodic patterns in the periodic hash table are neither closed nor parsimonious. Mining parsimonious periodic patterns, we should check two kinds of property. First, one is closed pattern mining in Lemma 7 and the second one is parsimonious pattern mining in Lemma 8 that means others do not subsume its periodicity. If patterns satisfy these two properties, we said it is parsimonious periodic pattern. At first, we prune non-closed patterns. After removing non-closed patterns, those patterns may be parsimonious or not. There is a main difference between closed and parsimonious periodic patterns. Closed periodic patterns are those patterns which have no super pattern with the same support or same pattern with large support. On the other hand parsimonious periodic patterns are those patterns which periodicities are not subsumed by other periodicity. In Table 3, we show first pattern generates two periodic descriptors which are closed and parsimonious because there is no supergraph with same support and periodicity. However, the second pattern one

Table 2
Periodic patterns and descriptors.

Patterns	Descriptors Information
	(1,1,2), (2,2,2)
	(1,1,3), (2,1,2), (1,2,2), (2,1,3), (3,1,2), (1,1,5), (1,2,3), (2,1,4), (4,1,2), (3,2,2), (3,1,3), (1,4,2), (2,3,2)
	(1,3,2)

CORRECTED PROOF

Table 3
Pruning non closed and non parsimonious periodic patterns.

Patterns	Descriptors	Closed	PPSE
			
	(1,1,2)	Y	Y
	(2,2,2)	Y	Y
			
	(1,1,3)	N(1,1,4)	
	(2,1,2)	N(2,1,3)	
	(1,1,4)	N(1,1,5)	
	(1,2,2)	N(1,2,3)	
	(2,1,3)	N(2,1,4)	
	(3,1,2)	N(3,1,3)	
	(1,1,5)	Y	Y
	(1,2,3)	Y	N(1,1,5)
	(2,1,4)	Y	N(1,1,5)
	(3,1,3)	Y	N(1,1,5)
	(1,4,2)	Y	N(1,1,5)
	(2,3,2)	Y	N (1,1,5)
	(3,2,2)	Y	N (1,1,5)
	(4,1,2)	Y	N(1,1,5)
			
	(1,3,2)	Y	Y

descriptor (1,1,5) subsumed all others descriptors and there is no supergraph with the same supports so it is closed and parsimonious only one descriptor value is (1,1,5). And third pattern is parsimonious and closed at periodicity (1,3,2).

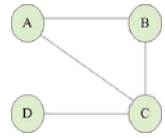
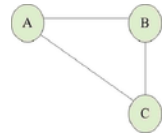
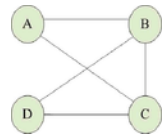
After pruning non-closed and non-parsimonious patterns we find parsimonious patterns, these are the output of our proposed *SPPMiner* algorithm shows in Table 4. Using descriptor (Des) information we find periodic patterns starting position (Start), period (Per), phase (Pha) and Support (Sup) value.

6. Experimental evaluation

We use three real-world dynamic social networks to evaluate our proposed *SPPMiner* algorithm as well as some inherent characteristics. We also use synthetic data to compare the performance of our algorithm with two existing algorithm *PSEMiner* (Lahiri & Berger-Wolf, 2010, 2008) and *ListMiner* (Apostolico, Barbares, & Pizzi, 2011; Barbares, 2010). For this comparison, these three algorithms are implemented in C^{++}. We implemented our proposed algorithm *SPPMienr* and implement *ListMiner* according to the pseudo code in Barbares (2010) and used *PSEMiner* source code available in Code (2010). The experiments are run on 3.3 GHz Intel Core i5 with 4GB RAM, in Windows 7.

These algorithms use Google dense/sparse hash library (Hash Library, 2015) which is more time and memory efficient. In all experiments, the reported computation time is the sum of the user and CPU time. Memory usage is the maximum resident set size reported by C^{++} memory usage function.

Table 4
Parsimonious periodic patterns.

Patterns	Des	Start	Per	Pha	Sup
					
	(1,1,2)	1	1	0	2
	(2,2,2)	2	2	0	2
					
	(1,1,5)	1	1	0	5
					
	(1,3,2)	1	3	1	2

6.1. Datasets

Dynamic social networks are collected from different sources and covering a range of interaction dynamics. These networks are described below.

6.1.1. Facebook wall post

We consider a Facebook dataset (Viswanath, Mislove, Cha, & Gummadi, 2009) gathered wall post information about 90,269 users September 26, 2006 to January 22, 2009. In total, 838,092 wall posts were observed, an average 13.9 wall posts per user. This covers communication between 188,892 distinct pairs of users. One-day quantization is measured as unique timestamps.

6.1.2. Reality mining

Cell phones with the nearness tracking technology were distributed to 100 students at the Massachusetts Institute of Technology over the course of an academic year (Eagle & Pentland, 2006). The timestamps quantization is chosen as one day.

6.1.3. Youtube

YouTube dataset (Mislove, Marcon, Gummadi, Druschel, & Bhattacharjee, 2007) is a video-sharing social networking data where users can create groups and share video information which other users can enjoy. The YouTube data were obtained on January 15, 2007 and consists of over 1.1 million users and 4.9 million links. User defined groups are created when groups of user can enjoy. In this dataset we consider each group is created at individual timestamps. We consider 5000 timestamps user-defined groups for our experiment.

6.1.4. Synthetic data

Synthetic datasets are used to better understand the performance comparison of these algorithms. The intention of these sets of experiments is to illuminate why and when our algorithm outperforms the others.

Starting with a population of 150 individuals, which creates 100,000 interactions, we generated a single graph of density at each timestamp. The edges of this graph represent interactions were then sampled independently at random for each of the T timestamps. Al-

UNCORRECTED PROOF

though this is not intended to be a realistic model of a social network, it allows us to control two parameters crucial to the mining process the overall density of the dynamic network, and the number of timestamps. Since real social networks are generally sparse, we used different densities of dynamic networks from 0.005 to 0.10. Density 0.005 means each timestamps 0.005*10000 = 50 interactions are active. These active interactions are randomly selected by using random function. The variation of synthetic data parameters for the synthetic networks expresses diversity characteristic of networks, including very low density (Ex-1.1 and Ex-1.2), medium density (Ex-1.3 and Ex-1.4) and high density (Ex-1.5 and Ex-1.6) are shown in Table 5. The experiments analyze will show network density is a parameter that has a significant influence on our *SPPMiner* algorithm.

6.2. Experimental time analysis

This section shows the execution time comparison among our proposed method and other two existing works. The execution times of three techniques are performed where minimum support $\sigma = 3$ and mining patterns are parsimonious.

The reality mining dataset is high density network. The number of vertices is low (100) and the number of timestamp is medium (544). *SPPMiner* algorithm generates periodic descriptors for each entity (interactions between two vertices) that why the low number of vertices interactions mining needs short time and less memory. Similarly, Ex-1.4 creates a sequence of 400 casual numbers between 1 and 10,000 and Ex-1.5 creates 700 numbers between 1 and 10,000. These networks are also dense. Therefore, these dense networks are changed from time to time defined by different graph structures. The probability of common subgraph computation between two graphs is high that reason *ListMiner* and *PSEMiner* needs more *MCS* computation.

The Fig. 5(a) shows that *SPPMiner* is faster than two existing works in Reality mining and YouTube datasets. In Reality dataset, our proposed method is two times faster than *ListMiner* and three times faster than *PSEMiner*. In YouTube dataset, the *SPPMiner* execution time is 63 s where the *PSEMiner* execution time is 180 s. That means our method is around three times faster than the *PSEMiner*. It is also faster than *ListMiner*. On the other hand, Facebook dataset is medium density and its time is slightly higher. Although in this case our algorithm performs better than *PSEMiner* but it is not good as *ListMiner* because it mines common patterns between 387 entities where our proposed process, traverse large time than 387 entities at each timestamps that's why it needs little bit large computation time. In the high-density context, *PSEMiner* is much slower than *SPPMiner* compare to low density context. Because *PSEMiner* builds periodic pattern tree using graphs and current graph traverse the entire tree node and find common subgraph. Since the dense graph, it needs more time to compute MCS between graph and tree nodes.

The execution time comparison among three methods on synthetic datasets is shown in the Fig. 5(b). It shows the execution time of low

Table 5
Parameters of various datasets.

Dataset	Time	V	E	AvgE
Reality	544	100	4900	0.025
Facebook	1563	46,951	193,337	0.002
YouTube	5000	1,134,890	2,987,624	0.0004
Ex-1.1	1000	150	10,000	0.005
Ex-1.2	1000	150	10,000	0.01
Ex-1.3	1000	150	10,000	0.02
Ex-1.4	1000	150	10,000	0.04
Ex-1.5	1000	150	10,000	0.07
Ex-1.6	1000	150	10000	0.08

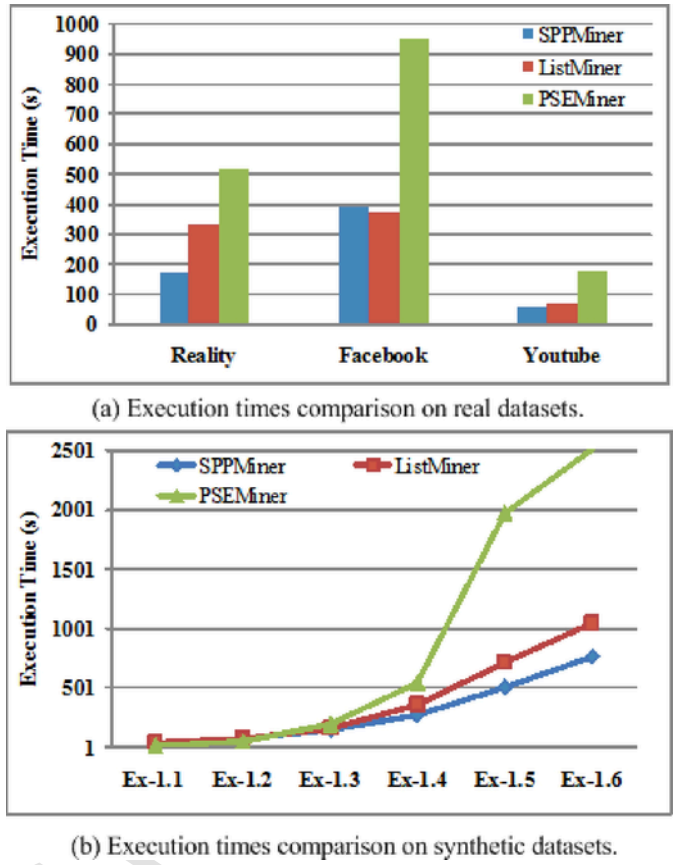


Fig. 5. Execution times comparison.

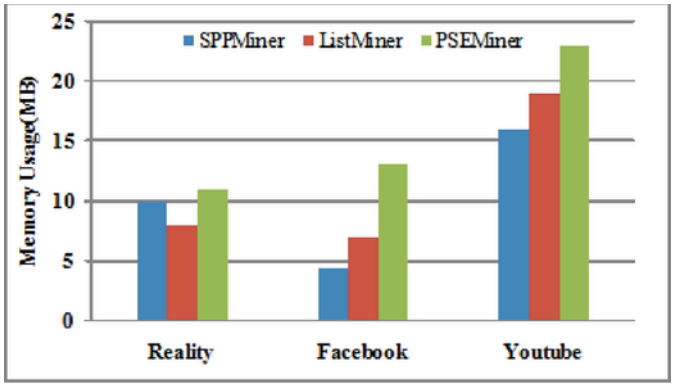
density based synthetic datasets Ex-1.1 and Ex-1.2 are almost same for the three methods. Another density dataset Ex-1.4 shows our proposed *SPPMiner* is around 1.5 times faster than *ListMimer* and around four times faster than *PSEMiner*. In the medium density context, analysis of Ex-1.3 reports that all these three methods execution times are almost same. Although, the theoretical time bound complexity of *SPPMiner* is better than *ListMiner* and *PSEMiner*. In dense network Ex-1.5 and Ex-1.6, our proposed method performs excellent than the others. According to above analyses, we can say that *SPPMiner* is time efficient when the network is medium or high density compare to low density.

6.3. Experimental space analysis

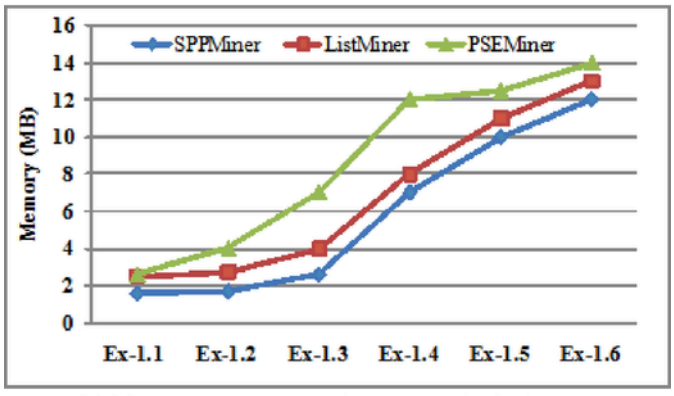
The analysis of the memory requirement of *SPPMiner, ListMiner* and *PSEMiner* are presented in this section. Fig. 6(a) shows the result's comparison of the memory usage of these algorithms with $\sigma = 3$. The *SPPMiner* use less memory in the Facebook dataset because it density is not too high and it creates less number of periodic behaviors. Reality dataset requires large memory because of each entity descriptor set generate $P^2_{\max}$ descriptors maximum that why it needs a large memory. YouTube dataset is memory efficient than the other methods. In conclusion, we said that our *SPPMiner* methods is memory efficient if the social networks total timestamp is greater than $P^2_{\max}$. In the synthetic dataset, Fig. 6(b) the *SPPMiner* uses less memory than the *ListMiner* and the *PSEMiner*.

This behavior can be justified by theoretical analysis of the space complexity. The space complexity of *PSEMiner* is $\left((V+E)\,N + P^2_{\max} + G\right)$ where N is the number of nodes in the

UNCORRECTED



(a) Memory usage comparison on real datasets.



(b) Memory usage comparison on synthetic datasets.

Fig. 6. Memory usage comparison.

tree, G is the number of descriptors, and V, E are the number of vertexes and edges, respectively. The space complexity of *ListMiner* is always $\left((V+E)\,T\,P_{\max}\right)$. Since the most part of the memory is used to store graphs, the dominant term of the space complexity expression in *PSEMiner* is $(V+E)\,N$. The space complexity of *SPPMiner* is $\left((V+E)\,P_{\max}^2\right.$. In dataset experiment 1.1, 1.2, 1.3 and 1.4 numbers of interactions (vertexes and edges) are 50, 100, 200 and 400 respectively. $P_{\max}$ is 40 so memory complexity of our process is less than existing methods *ListMiner* and *PSEMiner*.

6.4. Analysis of dense networks effects

In the previous section, we report the *SPPMiner* algorithm efficiency depend on network density. In section we will explain how density is affected our approach. Analyzing density effects, from Fig. 5(b) Ex-1.3 network density is 2% that time all these algorithms execution time is almost same. But Ex-1.4 network density is 4%, total number of interactions 10,000 and 400 are active at each timestamp. In this time our *SPPMiner* is 30% faster than *ListMiner* and 97% faster than *PSEMiner*. The Ex-1.5 is dense data that case *SPPMiner* is 43% faster than *ListMiner* and 292% faster than *PSEMiner*.

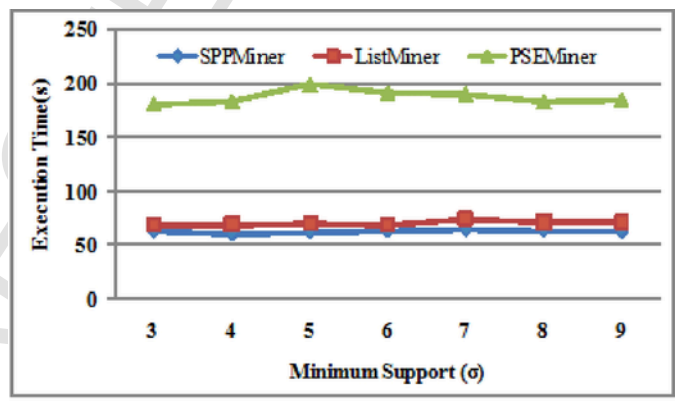
All these experiments networks continue 1000 timesteps and considered minimum threshold $\sigma = 3$ and $P_{\max} = 40$. Finally, we say that our approach is exceptionally faster than existing works in the medium and dense datasets.

6.5. Scalability analysis

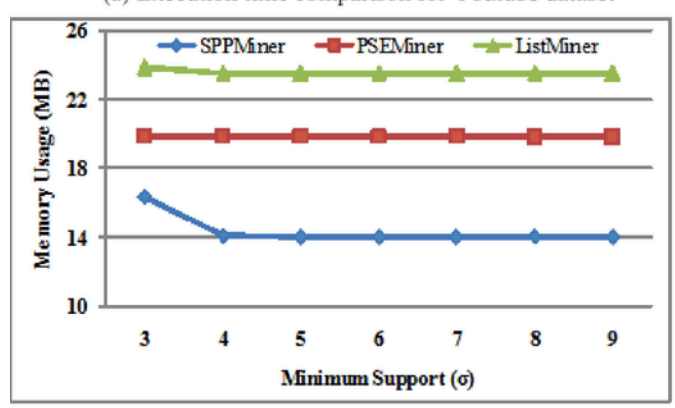
Execution time depends on minimum support and maximum period value. With the increases of maximum period, the execution time increases. On the other hand, with increase of minimum support the execution time reduces. Because when minimum support is small, each algorithm produces gigantic periodic patterns and finds large number of parsimonious patterns. Mining these patterns need tremendous time. The given Fig. 7(a) depicts that proposed *SPPMiner* is three times faster than *PSEMiner* and its execution time is almost same for different minimum supports. According to this figure, it also outperforms than the *ListMiner*. The Fig. 7(b) shows the memory usage of different parsimonious periodic pattern mining algorithm. When minimum support is three *SPPMiner* produce gigantic patterns, takes a little bit more memory than the other times of minimum support value. We can see the memory usage of our proposed method is small compare than the other methods.

Figs. 8(a) and 9(a) show that scalability analysis of the reality mining dataset and synthetic datasets Ex-1.4 respectively. In this case, at minimum support is three and it requires more time compare to others value.

Fig. 9(a) shows that the minimum support scalability where $P_{\max}$ = 50. And Fig. 9(b) shows that the scalability analysis of our proposed *SPPMiner* as well as *ListMiner* and *PSEMiner*. Our propose *SPPMiner* is more scalable than two existing works because when $P_{\max}$ increase *ListMiner* and *PSEMiner* create excessive nodes that



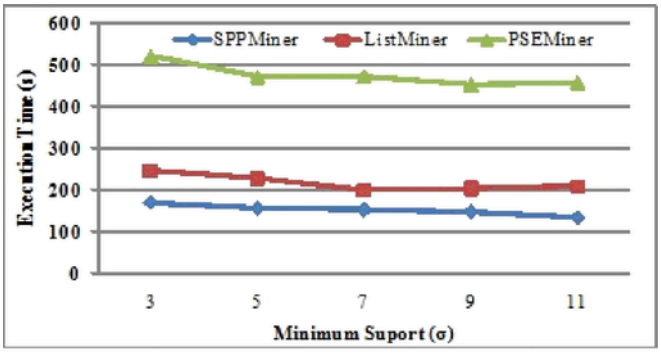
(a) Execution time comparison for Youtube dataset



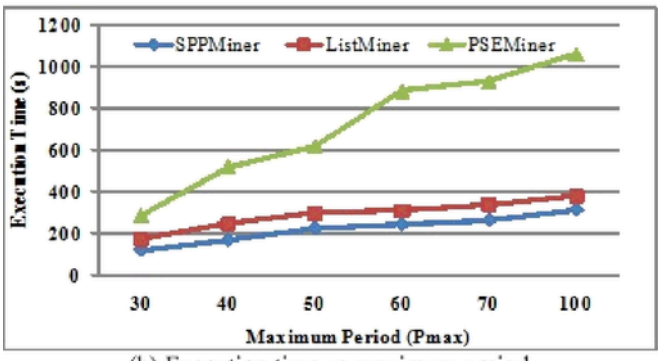
(b) Memory Usage comparison for Youtube dataset

Fig. 7. Performance analysis of YouTube dataset.

UNCORRECTED PROOF



(a) Execution time vs minimum support



(b) Execution time vs maximum period

Fig. 8. Scalability test for reality mining dataset.

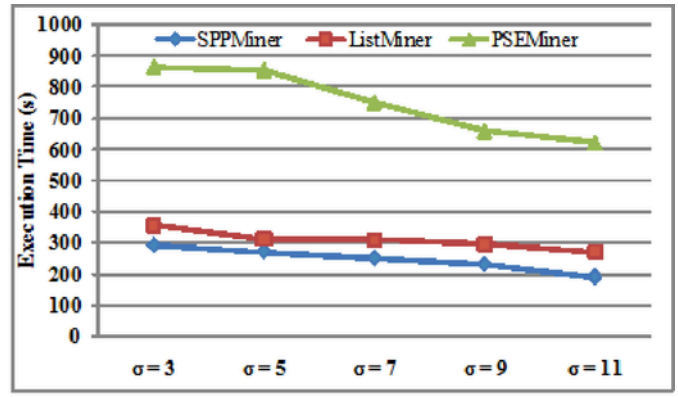
take more comparison which is time consuming. In this experiment, we use artificial data Ex-1.4 and minimum support = 3.

6.6. Analysis of parsimonious periodic patterns

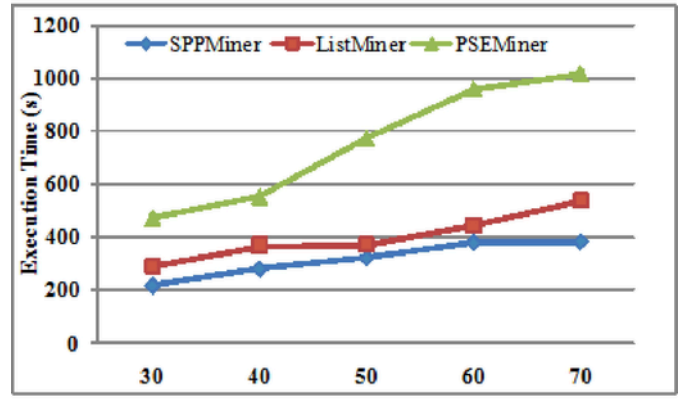
In this section, the analysis of parsimonious periodic patterning of the *SPPMimer* algorithm is reported. This analysis is performed on two tracks. First one is number of periodic patterns vs. support and second one is number of periodic patterns vs. period.

Fig. 10 shows the number of parsimonious periodic patterns (y axis) for each real dataset based on support value (x-axis). It has been shown that dense dataset reality mining produces a large number of parsimonious periodic patterns more than 10,000 though its total timestamp is only 544. With the increases of support its pattern number decreases rapidly. Facebook dataset produces vast data at support 3,4,5 that represent most of the pattern are within support 3 to 5 and then it reduces very rapidly and at the end support 14 it produces only 28 patterns. On the other hand, YouTube dataset produces a small number of periodic patterns. It generates 469 patterns when minimum support is 3. Then the pattern's numbers reduce sharply and reach 127 when minimum support is 4 after that its number of patterns decreases gradually. All these experiments run based on period $P_{\max} = 40$.

Fig. 11 shows the number of parsimonious periodic patterns (y axis) for each artificial dataset based on the different period value (x-axis). It has been shown that the Facebook dataset produces a large number of parsimonious periodic patterns and generating patterns are decreasing with the increases of period. In YouTube dataset, we get 128 patterns for period 1 and it reduces with the increase of periods. All these patterns are mined based on minimum support $\sigma = 3$. Experiments analyses showed that the highest number of parsimonious periodic patterns and the highest values of support occur on high-density networks.



(a) Execution time vs minimum support



(b) Execution time vs maximum period

Fig. 9. Scalability test for synthetic (Ex-1.4) dataset.

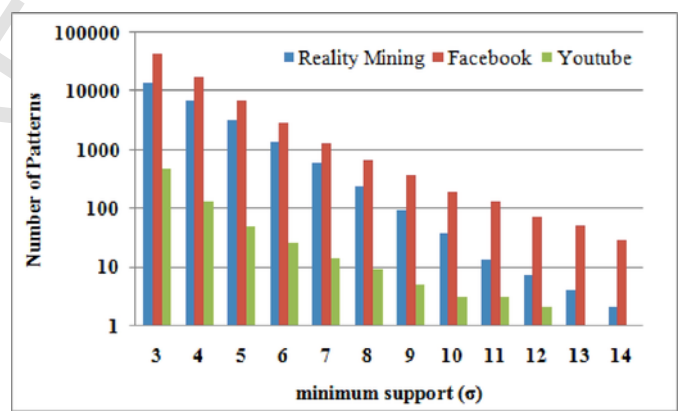


Fig. 10. Parsimonious periodic patterns vs. minimum support.

6.7. Knowledge discovery

In the Fig. 12(a) and (b) has been shown that Facebook dataset contains pair to pair communication that are weekly and continue up to 7 weeks and another one is daily and 2 day interval wall post communication continue up to 10 times. In the YouTube dataset we get some significant relationship. There exists one group with 49 members which contain 3 times communications at sixty days intervals. From this kind or relationship, we suppose that they are classmates and enjoy video together.

UNCORRECTED

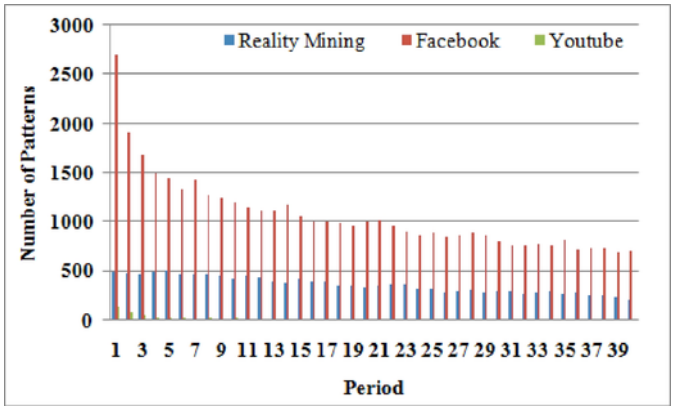


Fig. 11. Parsimonious periodic patterns vs. period value.

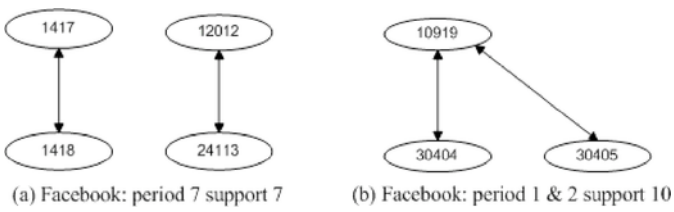


Fig. 12. Inherent patterns from Facebook.

In this section, we have shown the effectiveness and efficiency of our proposed *SPPMiner*. Experimental space and time analysis show that our method is significantly efficient for medium and dense dynamic networks and outperform the existing algorithm in both execution time and memory usage. We also mine some interesting periodic patterns from real datasets that represent very informative information.

7. Discussion

Our proposed algorithm can mine periodic patterns among a set of patterns in dynamic networks. The mined periodic patterns help detect the important relation among entities which have deep impact on not only in the relation but also in various other real-life applications (like as business and advertisement) and on predicting future phenomena.

In real life, merchants used to advertise their merchandise to the consumer based on the consumer choices. Most often, telecasting and making advertisement are too costly. On the other hand, if merchants cannot make their product advertisements to customers, their business will face serious challenges. In this case our proposed algorithm thought to be of one of the best solutions.

The periodic pattern mined in our proposed approach is also impactful in various challenging real-life application domains such as machine learning, bio-informatics, trajectories, social media networks, statistics and lots more. This section deals with some real-life applications of the stated domains.

Now-a-days human periodic behavior prediction and analysis appear as interesting research areas. In addition, in conjugal life, it is very significant to realize spouse's behavior. If someone is unable to predict or understand his/her partner's behavior, especially periodic behavior, conjugal life may be threatened. On the other hand, sometimes brides and bridegrooms take some events seriously though it may not be that important to other. In this case unhappiness might engulf their relationship. In fact, through our proposed technique in human behavior, we can predict the human nature and attitude that is more significant in conjugal life.

In the domain of medical science, diagnosing diseases depends on periodic behavior. In the treatment of mental disease, it is more important because patient's behaviors change time to time. If we find some periodic behavior, it would be very helpful to diagnosing the disease. Also, we can apply our method to predict the exchange of market cost effect analysis and can help to propose different companies' new products based on consumers periodic behaviors.

8. Conclusions

To exploit the motivation to periodic behaviors, the paper proposed a supergraph based periodic pattern mining algorithm called *SPPMiner*. This algorithm is polynomial unlike most graph mining algorithms. We proposed two efficient pruning methods: non-closed pattern pruning and non-parsimonious periodicity pruning methods which reduce the computational cost of finding these patterns. The experimental result shows that the proposed method is more time and memory efficient than the other two algorithms to discover periodic patterns for the synthetic datasets and real life datasets. We also found some inherent relationships that indicate strongly connected groups of users in dynamic networks.

In our current study, we did not consider noisy data and the time-shifts of events. For monthly event or news portals, it is published at a specific date of months, even though each month has different number of days. We did not consider those maximum period equivalent of total dynamic time that results some significant events may be discarded. Periodic patterns that occur sequentially convey significant meaning.

In our future work, we will consider noisy data, including time-shifts events. We will develop efficient method for mining sequential periodic patterns in dynamic social networks. Recurring periodic patterns in time series, item products and trajectory datasets are not exploited yet, we will exploit recurring periodic pattern in these domains.

Uncited references

Yang, Hong, Chen, and Lan (2013) and Nishi, Ahmed, Samiullah, and Jeong (2013).

Acknowledgment

The authors are grateful to the anonymous reviewers for their comments that improved the quality of this paper. This research was supported by the MSIP, Korea, under the G-ITRC support program (IITP-2016-R6812-16-0001) supervised by the IITP.

References

Agrawal, R., Imieliński, T., Swami, A., 1993. Mining association rules between sets of items in large databases. ACMSIGMOD record. 22. ACM, 207–216.
Apostolico, A., Barbares, M., Pizzi, C., 2011. Speedup for a periodic subgraph miner. Information Processing Letters 111 (11), 521–523.
Barbares, M. (2010). Periodic subgraph mining in dynamic networks.
Chapanond, A., Krishnamoorthy, M.S., Yener, B., 2005. Graph theoretic and spectral analysis of enron email data. Computational & Mathematical Organization Theory 11 (3), 265–281.
Code (2010). http://compbio.cs.uic.edu/software/periodic/.
Diesner, J., Carley, K.M., 2005. Exploration of communication networks from the enron email corpus. SIAM international conference on data mining: Workshop on link analysis, counterterrorism and security, Newport Beach, CA. Citeseer.
Eagle, N., Pentland, A., 2006. Reality mining: Sensing complex social systems. Personal and Ubiquitous Computing 10 (4), 255–268.

UNCORRECTED PROOF

Fischhoff, I.R., Sundaresan, S.R., Cordingley, J., Larkin, H.M., Sellier, M.-J., Rubenstein, D.I., 2007. Social relationships and reproductive state influence leadership roles in movements of plains zebra, equus burchellii. Animal Behaviour 73 (5), 825–831.

Han, J., Dong, G., Yin, Y., 1999. Efficient mining of partial periodic patterns in time series database. Data engineering, 1999. Proceedings., 15th international conference on. IEEE, 106–115.

Hash Library, G. (2015). http://code.google.com/p/google-sparsehash/, version 2.2.

Huang, K.-Y., Chang, C.-H., 2004. Asynchronous periodic patterns mining in temporal databases. Proceedings of the IASTED international conference on databases and applications (DBA). 43–48.

Juang, P., Oki, H., Wang, Y., Martonosi, M., Peh, L.S., Rubenstein, D., 2002. Energy-efficient computing for wildlife tracking: Design tradeoffs and early experiences with zebranet. ACMSigplan notices. 37. ACM, 96–107.

Lahiri, M., Berger-Wolf, T.Y., 2008. Mining periodic behavior in dynamic social networks. Data mining, 2008. ICDM'08. Eighth IEEEinternational conference on. IEEE, 373–382.

Lahiri, M., Berger-Wolf, T.Y., 2010. Periodic subgraph mining in dynamic networks. Knowledge and Information Systems 24 (3), 467–497.

Ma, S., Hellerstein, J.L., 2001. Mining partially periodic event patterns with unknown periods. Data engineering, 2001. Proceedings. 17th international conference on. IEEE, 205–214.

Mislove, A., Marcon, M., Gummadi, K.P., Druschel, P., Bhattacharjee, B., 2007. Measurement and analysis of online social networks. Proceedings of the 5th ACM/Usenix internet measurement conference (IMC'07), San Diego, CA.

Nishi, M.A., Ahmed, C.F., Samiullah, M., Jeong, B.-S., 2013. Effective periodic pattern mining in time series databases. Expert Systems with Applications 40 (8), 3015–3027.

Ozden, B., Ramaswamy, S., Silberschatz, A., 1998. Cyclic association rules. Data engineering, 1998. Proceedings., 14th international conference on. IEEE, 412–421.

Tanbeer, S.K., Ahmed, C.F., Jeong, B.-S., Lee, Y.-K., 2009. Discovering periodic-frequent patterns in transactional databases. Advances in knowledge discovery and data mining. Springer, 242–253.

Viswanath, B., Mislove, A., Cha, M., Gummadi, K.P., 2009. On the evolution of user interaction in facebook. Proceedings of the 2nd ACM workshop on online social networks. ACM, 37–42.

Wasserman, S., Faust, K., 1994. Social network analysis: Methods and applications. 8. Cambridge university press.

Yang, J., Wang, W., Yu, P.S., 2003. Mining asynchronous periodic patterns in time series data. Knowledge and Data Engineering, IEEE Transactions on 15 (3), 613–628.

Yang, J., Wang, W., Yu, P.S., 2004. Mining surprising periodic patterns. Data Mining and Knowledge Discovery 9 (2), 189–216.

Yang, K.-J., Hong, T.-P., Chen, Y.-M., Lan, G.-C., 2013. Projection-based partial periodic pattern mining for event sequences. Expert Systems with Applications 40 (10), 4232–4240.

Yin, Z., Cao, L., Han, J., Zhai, C., Huang, T., 2011. Lpta: A probabilistic model for latent periodic topic analysis. Data mining (ICDM), 2011 IEEE 11th international conference on. IEEE, 904–913.

Zheng, H.-T., Jiang, Y., 2012. Towards group behavioral reason mining. Expert Systems with Applications 39 (16), 12671–12682.

UNCORRECTED PROOF